🐶 Mamás Perrunas - Juntos es mejor 🐾

1. Enlaces **1**
2. Introducción. **1**
3. Descripción ¿Qué necesidad cubre nuestro proyecto? **2**
4. Funciones y rendimientos deseados **3**
4. Tecnologías Utilizadas **3**
4.1 ¿Qué aporta al proyecto usar componentes standalone? 3
5. Experiencia de usuario (UX). **4**
5.1 Elección de patrones de diseño. 5
5.2 Elección de colores. 6
5.3 Uso de imágenes. 6
5.4 Facilidad de uso y navegación. 7
6. Arquitectura del proyecto **8**
6.1 Estructura y organización general. 8
6.2 Estructura de la carpeta app 9
6.3 Organización y relación de los componentes. 11
7. Estructura y funcionamiento de la lógica. **13**
7.1 Uso de datos .json en la carga de productos. 13
7.2 Uso de localStorage para la lógica de registro. 14
7.3 Uso de sessionStorage para el login component. 15
7.4 Funcionamiento de la api implementada. 16
8. Conclusión **17**

1. Enlaces

Página web

Enlace git

2. Introducción.

El proyecto Mamás Perrunas nace con el objetivo de crear una plataforma integral para familias con perros, proporcionando información, servicios y una comunidad interactiva para los amantes de los animales. La web no solo busca ser un punto de referencia en el mundo canino, sino que también responde a una creciente necesidad social: en España, el número

de perros supera al de niños, lo que evidencia la importancia de los animales en los hogares.

El desarrollo del proyecto se ha basado en tecnologías modernas como **Angular con componentes Standalone**, garantizando una estructura modular, eficiente y de alto rendimiento. Además, se ha priorizado la **experiencia de usuario (UX)** con una navegación intuitiva, un diseño visual atractivo y elementos optimizados para garantizar un acceso fluido a la información.

La plataforma cuenta con una tienda online afiliada a Amazon, un blog informativo, un sistema de registro y autenticación mediante `localStorage` y `sessionStorage`, así como una sección de eventos y servicios para profesionales del sector. También se integra una API para obtener imágenes y datos sobre razas de perros, mejorando la funcionalidad y atractivo del sitio.

3. Descripción ¿Qué necesidad cubre nuestro proyecto?

Mamás Perrunas aspira a ser mucho más que una simple página web; su objetivo es convertirse en el portal de referencia para todos los amantes de los perros. Diseñado para ofrecer un servicio integral a las familias con mascotas, su misión es reflejar y responder a la visión actual de los animales de compañía como un miembro más del hogar.

Según los datos más recientes, en España hay aproximadamente 9,3 millones de perros, lo que equivale a un perro por cada 5,25 habitantes. Además, en 2024 se estima que el 49% de los hogares en España tiene al menos una mascota, y el 93% de las personas considera a sus mascotas un miembro más de la familia (AEDPAC, 2020). Este fenómeno ha dado lugar al concepto de familias multiespecie, inter-especie o trans-especie, términos que reflejan la creciente integración de los animales en la vida cotidiana de los hogares.

Un dato revelador es la comparación entre la cantidad de perros y niños en España. Actualmente, hay 9,3 millones de perros, mientras que la población infantil asciende a 6,6 millones de niños. Esta tendencia refleja cambios demográficos y sociales que evidencian la importancia de los perros en la sociedad y resalta el enorme potencial de este mercado.

4. Funciones y rendimientos deseados

La **Tienda Online** está vinculada a Amazon a través de su sistema de afiliados, lo que permite monetizar la página sin necesidad de gestionar stock ni logística. Esto ofrece varias ventajas, como una inversión inicial baja, ingresos pasivos mediante comisiones generadas por compras realizadas a través de los enlaces de afiliados, acceso a una gran variedad de productos sin preocuparse por el inventario y la confianza que brinda Amazon como una plataforma reconocida.

En cuanto a las secciones en desarrollo, la de **Eventos** estará enfocada en la organización y promoción de actividades relacionadas con el mundo canino. Por otro lado, el área de **Servicios** permitirá a los profesionales del sector ofrecer sus servicios directamente a los usuarios interesados. Además, la **Comunidad**, actualmente en construcción, se convertirá en un espacio para dueños de perros donde podrán compartir experiencias, resolver dudas y acceder a "El experto responde", una sección en la que especialistas registrados ofrecerán asesoramiento.

El **Blog** será una fuente de información organizada en distintas categorías para ofrecer contenido de interés a los usuarios. Dentro de este, la sección /razas contará con un buscador que permitirá conocer diferentes razas de perros y filtrarlas según su tamaño.

4. Tecnologías Utilizadas

Frontend: Angular (Standalone)

TypeScript

CSS para la definición de estilos

Documentación: Compodoc

4.1 ¿Qué aporta al proyecto usar componentes standalone?

Todos los componentes que hemos usado son standalone ¿por qué? Angular por defecto desde su versión 14 (2022) precisamente la introducción de estos componentes standalone fue la mejora fundamental de esta versión. Los componentes standalone permiten crear aplicaciones sin la necesidad de módulos (NgModule). Es decir, ya no existe el módulo module.ts. Los módulos desaparecen y los componentes pasan a poder ser importados directamente sin necesidad de declararlos en un módulo. Para definirlos, basta con agregar la propiedad standalone: true en el decorador @Component.

```
 */
@Component({
  selector: 'app-bannerportada',
  standalone: true,
  templateUrl: './bannerportada.component.html',
  styleUrls: ['./bannerportada.component.css']
})
```

Por ejemplo si montamos una landing page usando componentes no standalone tendríamos que crear un archivo landing.module.ts donde se declaren e importen todos los componentes que usa la landing page dentro de un Ngmodule y luego este module declararlo en app.module.ts o bien hacerlo directamente en app.module.ts pero en proyectos grandes deberíamos de modularlos en módulos independientes.

Esta necesidad de declarar los componentes en un módulo desaparece si usamos componentes standalone. Y gracias a ello ya no es necesario importar el módulo completo si no sólo el componente que se vaya a usar haciendo mucho más eficaz la carga de la aplicación pues si por ejemplo hemos declarado todos los componentes en app.module.ts, entonces Angular carga todos al iniciar la aplicación, sin importar qué página esté viendo el usuario lo cual sin duda hará mucho más lenta nuestra web.

En resumen, las ventajas de usar componentes standalone son evidentes: reducen la cantidad de código, nos simplifican la reutilización de componentes y mejoran significativamente el rendimiento y velocidad de carga de la página.

5. Experiencia de usuario (UX).

La experiencia del usuario (UX) se refiere a la forma en que un usuario interactua con una web y a la sensación que tiene al hacerlo. Un buen UX asegurará que los usuarios accedan

a lo que necesitan de manera intuitiva y rápida y de una manera agradable, maximizando la usabilidad, accesibilidad y la eficacia del sitio.

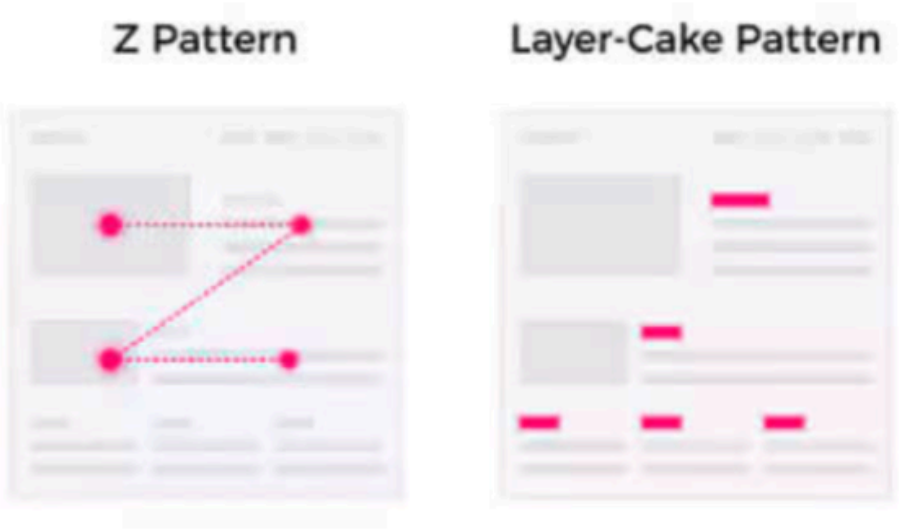
Nuestro enfoque UX se basa en tres principios clave:

✔ Facilidad de uso y navegación → El sitio debe permitir un desplazamiento intuitivo.

✔ Buen diseño visual → Se utilizan elementos visuales atractivos con mínimo texto en la portada y elementos de llamada a la acción.

✔ Cargas breves y diseño lógico → Secciones claramente delimitadas y rendimiento optimizado.

5.1 Elección de patrones de diseño.



Estos son los dos patrones que vamos a usar en nuestra web alternandolos según las páginas y el objetivo perseguido en cada una de ellas.
El patrón en Z es ideal para diseños con menor cantidad de texto y más elementos visuales como por ejemplo la landing page.
El patrón de pastel en capas lo usaremos en secciones como el blog donde la mayor de contenido es mayor pues garantiza secciones bien definidas que ayudan al usuario a acceder a la información que busca identificando secciones y encabezados.

5.2 Elección de colores.

Esta paleta de colores puede definirse como una combinación cálida y natural, con tonos que evocan sensaciones de energía, naturaleza y vitalidad. Principalmente usaremos colores como el verde y el naranja aunque no dejaremos de lado un color alegre y positivo como el amarillo en todas sus gamas.

El verde es un color que representa naturaleza, bienestar, frescura y tranquilidad. Dado que los perros suelen asociarse con la naturaleza y la vida al aire libre, este color puede evocar sentimientos de conexión con el entorno natural y bienestar. También induce una sensación de calma y equilibrio, lo cual es beneficioso para crear un ambiente relajado y amigable en la web.

El naranja es un color que transmite energía, calidez, comunidad y confianza. Es un color vibrante que puede ayudar a estimular la interacción y el dinamismo en la comunidad. También se asocia con la creatividad y la alegría, lo que puede hacer que los usuarios se sientan bienvenidos y motivados a participar.

Es un color que representa acción con lo que sería perfecto para botones que nos interese sean pulsados como comprar o suscribirse.

5.3 Uso de imágenes.

Muchas veces utilizamos imágenes en nuestra web sin pensar en las implicaciones legales, de derechos de autor o en cómo afectan el rendimiento del sitio. Por eso en mamás perrunas hemos optado por imágenes generadas por inteligencia artificial.

¿Qué ventajas nos ofrece el uso de estas imágenes? En primer lugar, elimina el riesgo de infringir licencias o derechos de autor, ya que las imágenes creadas son completamente originales y no hay limitaciones sobre cómo, dónde o cuántas veces puedes usarlas ni es necesario preocuparse de atribuciones.

Además, permite mantener una coherencia visual en toda la web, asegurando un estilo unificado sin depender de múltiples fuentes externas reduciendo costes de bancos de imágenes o fotógrafos y obteniendo imágenes únicas en cuestión de segundos.

En definitiva, el uso de imágenes generadas por IA garantiza un diseño único, optimizado y sin preocupaciones legales o de rendimiento, haciendo que la web luzca profesional y eficiente. No obstante y aunque la IA puede generar imágenes realmente válidas carece de la intuición, la sensibilidad artística y la capacidad de interpretar emociones y conceptos de manera profunda que sí nos puede aportar un diseñador. Así que si nuestro presupuesto lo permite lo ideal es combinar ambas herramientas.

5.4 Facilidad de uso y navegación.

Para garantizar que los visitantes encuentren lo que buscan de manera eficiente, hemos incorporado varios elementos clave.

El menú superior permanece presente permitiendo un acceso rápido a las secciones principales del sitio sin necesidad de desplazarse hacia arriba.
Un Menú lateral en la sección del blog que facilita la navegación por categorías, artículos relacionados y contenido relevante de manera cómoda.

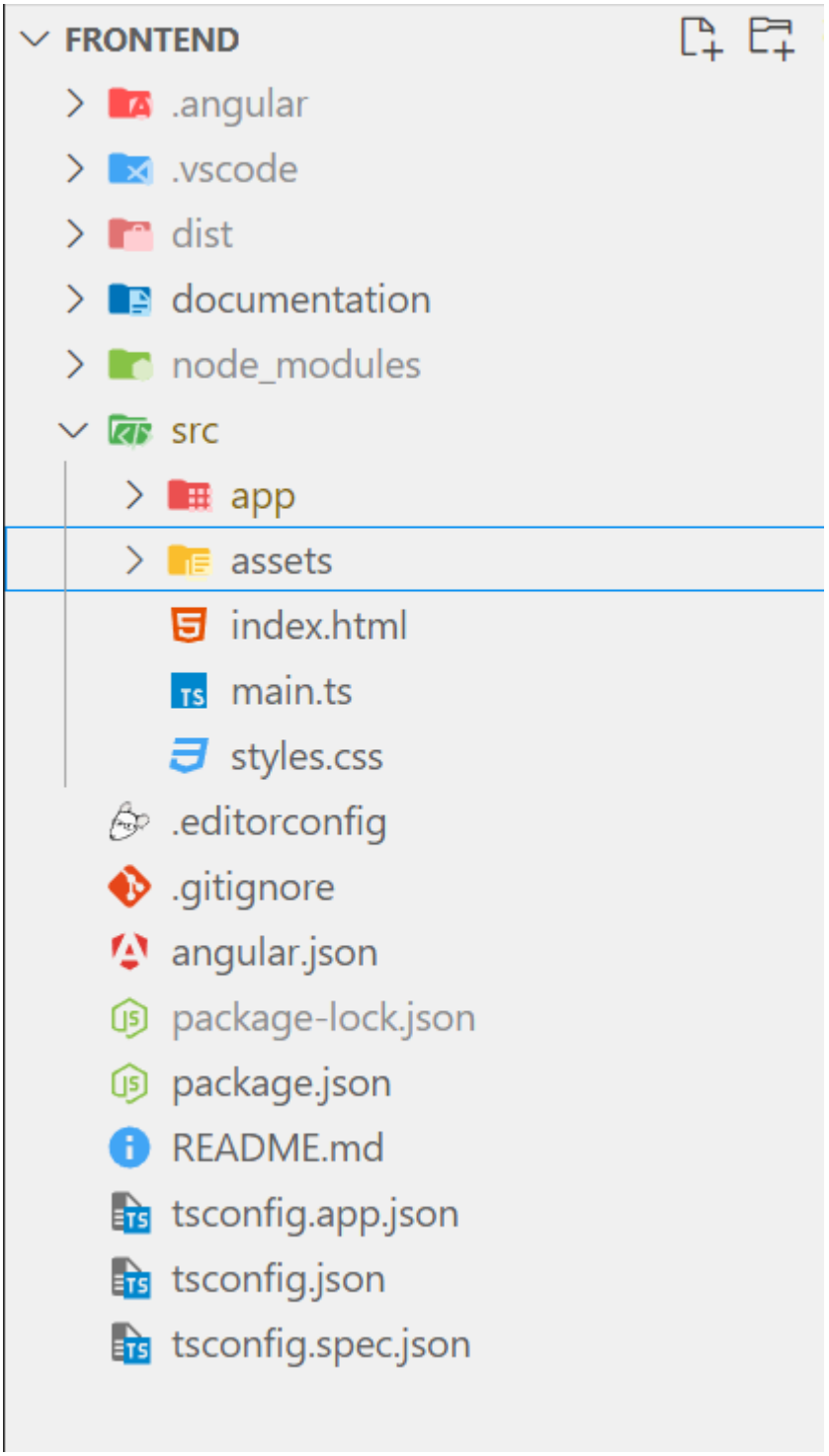
Migas de pan que ayudan a los usuarios a comprender su ubicación dentro del sitio y permiten regresar a secciones anteriores sin complicaciones.

Estos elementos contribuyen a una experiencia fluida, organizada y fácil, asegurando que los usuarios puedan moverse por el sitio de manera intuitiva y eficiente.

6. Arquitectura del proyecto

6.1 Estructura y organización general.

Ahora vamos a analizar la estructura de nuestro proyecto



src (source) —---------contiene el código fuente de la aplicación: componentes, páginas y servicios.

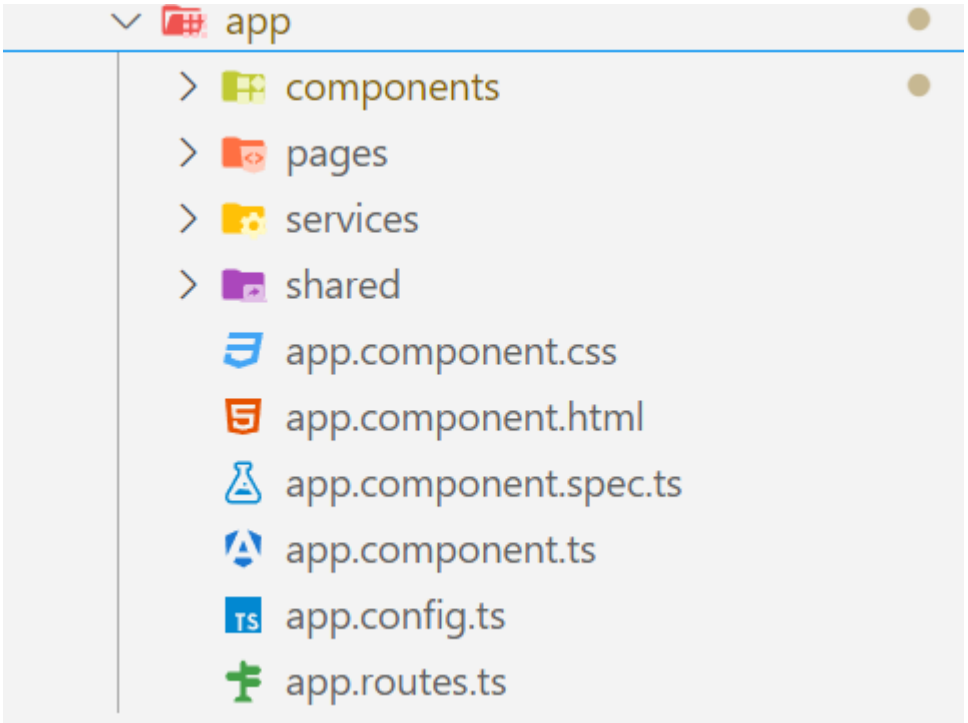
assets—----------contiene los elementos estáticos tales como imágenes o archivos .json

index. html—--------es el archivo fundamental donde angular carga la aplicación. Contiene un <app-root></app-root>, que es donde Angular renderiza la aplicación es decir la muestra en pantalla.

main.ts -- ---------punto de entrada en la aplicación. Llama a bootstrapApplication() para iniciar el proyecto.

styles.css -- --------- se establecen estilos comunes que se van a usar en todo el proyecto.

6.2 Estructura de la carpeta app



En componentes hemos definido todos los componentes que luego vamos a reutilizar en las distintas páginas. En el siguiente punto analizamos todos ellos y la relación que mantienen.

En page montamos la página importando los componentes que queremos usar en cada una de ellas.

En la carpeta service declaramos los servicios necesarios para el funcionamiento de nuestra aplicación; en nuestro caso el servicio que llama a la dog api que nos aporta las fotos de los perros.

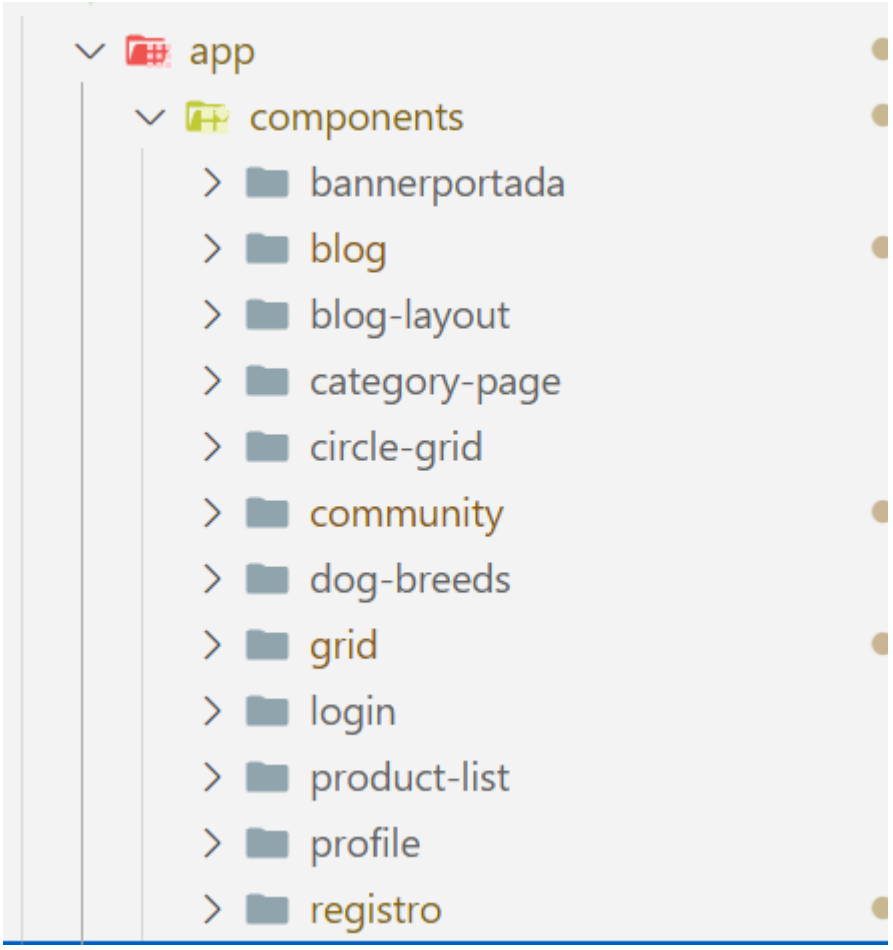
Y la carpeta shared recoge los componentes que van a ser reutilizados en todas las paginas como la navbar o el footer.

Luego vemos unos archivos fundamentales.

El archivo app.component.ts es el componente raíz de la aplicación en Angular. Su función principal es estructurar la interfaz de la aplicación y definir qué elementos se renderizan en todas las páginas. En concreto nuestro archivo muestra la barra de navegación en todas las páginas, renderiza dinámicamente la página según la ruta activa y muestra el pie de página en todas las páginas. Y esto lo hace siguiendo la estructura del html y aplicando estilos del archivo css si se ha implementado alguno para los componentes que se hayan recogido en el archivo html que en nuestro caso no lo hemos hecho.

El archivo app.routes.ts es crucial porque permite la navegación entre las diferentes páginas sin necesidad de recargar la web. Sin este archivo la aplicación no podría manejar múltiples páginas.

6.3 Organización y relación de los componentes.



Aunque en el compodoc podemos ver cada componente y analizar su estructura en este punto vamos a analizar de una forma esquemática la organización de nuestros componentes en relación con las funcionalidades de nuestra web.

Landing page

-Banner portada: titulo (H1) y foto

-Grid: diseño de cuadrículas

-Circle-grid

-Blog

-Community

Tienda

-Product-list: página inicial. Carga 3 productos aleatorios y categoría

-Category page: página de productos de cada categoría

Dog-api

-Dog-breeds: obtiene los datos de dog-api.service.ts

-Dog-list (page): muestra datos de dog-api

Registro-login-profile

-Guarda y recupera datos de local storage

7. Estructura y funcionamiento de la lógica.

Vamos a analizar dos puntos claves en la lógica de nuestro proyecto: carga de productos y registro en base de datos local.

7.1 Uso de datos .json en la carga de productos.

En este apartado analizaremos por qué hemos optado por almacenar los productos en un archivo externo en formato JSON en lugar de un array puro dentro del código TypeScript, las ventajas y desventajas de esta decisión, cómo se implementa esta lógica en nuestra aplicación y cuáles son los aspectos clave en este tipo de programación.

¿Qué ventajas nos aporta usar un archivo .json?

✅ Separación de datos y lógica: Mantiene los datos estructurados y separados del código de la aplicación, mejorando la organización.

✅ Facilita la escalabilidad: Si en el futuro queremos agregar más productos o categorías, solo es necesario modificar el archivo JSON sin tocar el código de la aplicación.

✅Nos permite organizar los productos en categorías sin la necesidad de recorrer todo el array para filtrar una determinada categoría. La estructura en datos .json nos permite obtener una determinada categoría de forma directa con el método find().

✅ Reutilización y portabilidad: Un archivo JSON puede ser reutilizado en otros sistemas o migrado fácilmente a una API sin modificar el código del frontend por ejemplo a una base de datos y también ser sustituido posteriormente por los datos que nos daría la propia api de amazon cuando la consigamos.

✅ Simplicidad en la gestión de datos: Nos permite trabajar con un formato estándar y ampliamente utilizado para el intercambio de información entre sistemas.

¿Cuál es la diferencia fundamental? JSON se carga desde una petición HTTP, lo que puede generar una pequeña latencia en comparación con los arrays embebidos en el código. Esta llamada la tenemos en el componente product.list y en category page.

7.2 Uso de localStorage para la lógica de registro.

Dado que en esta fase del proyecto hemos decidido no utilizar un backend, hemos buscado soluciones alternativas que nos permitieran mantener la funcionalidad esencial sin comprometer el desarrollo futuro. Uno de los aspectos clave de la aplicación es el sistema de registro e inicio de sesión de usuarios.
Para esta primera implementación, hemos optado por utilizar la memoria local del navegador (Local Storage y Session Storage) como mecanismo de almacenamiento temporal.

Si bien este método nos permite gestionar el registro e inicio de sesión de manera sencilla y rápida, es importante destacar que no es una solución definitiva, ya que presenta varias limitaciones en términos de seguridad y persistencia. No obstante, nos permite simular el comportamiento del sistema y establecer la base de la lógica de autenticación que en futuras fases se integrará con un backend real.

A continuación, analizamos los tres componentes involucrados en el registro, login y gestión del perfil de usuario.

✅**RegistroComponent**. Recoge las validaciones para un registro correcto mediante FormBuilder y Validators de Angular. En concreto, recoge tres validaciones .

El nombre debe tener un mínimo de dos caracteres y un máximo de 50. Es un campo obligatorio.

El email debe tener un formato válido (validator.email) y es también un campo obligatorio.

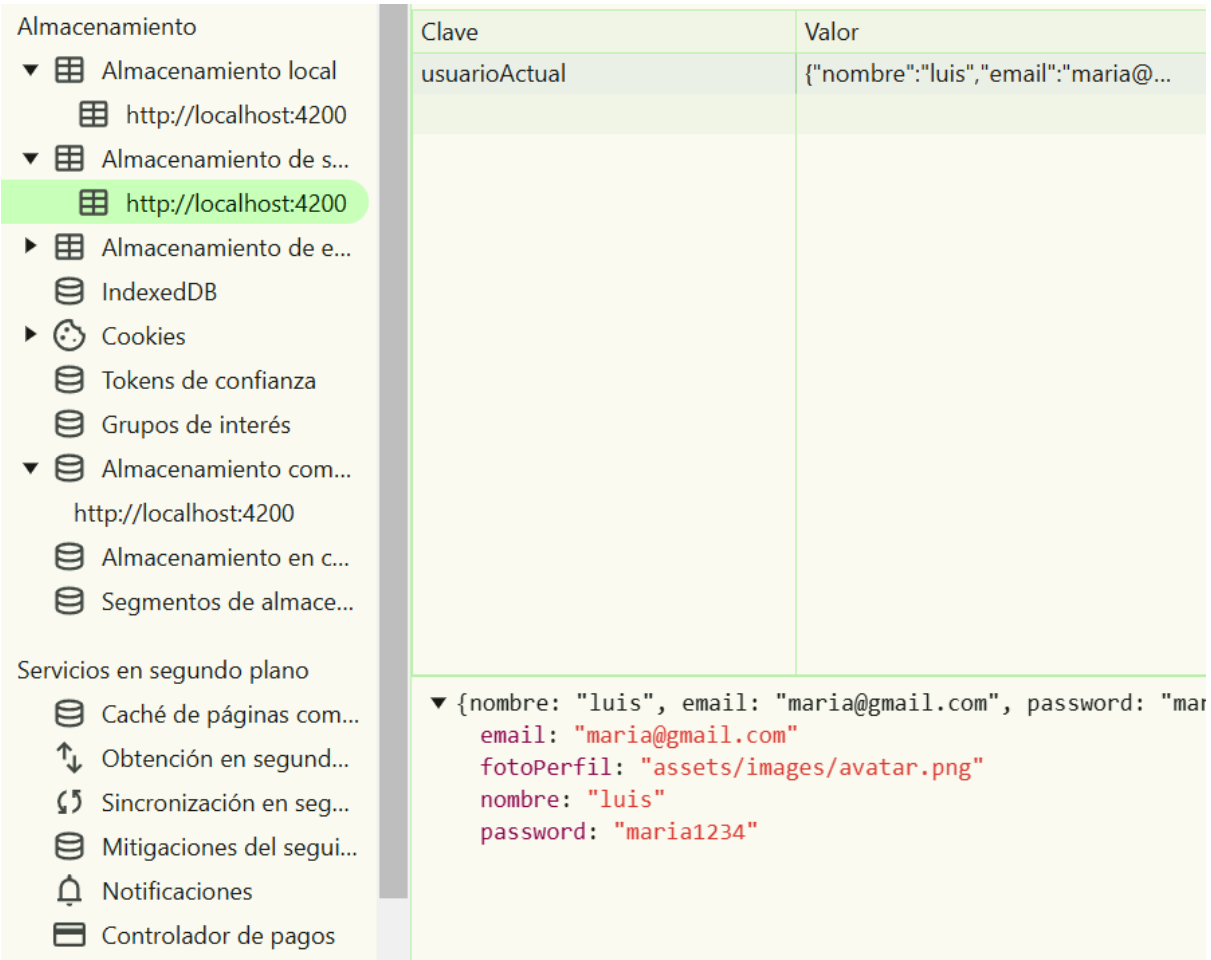
La contraseña debe tener mínimo seis caracteres y es también obligatorio.

Una vez que el usuario ha completado el formulario correctamente el método onSubmit () verifica que ninguno de los usuarios almacenados tenga el mismo email, le asigna una imagen predefinida para el perfil, y guarda los datos en formato json en local storage. Se permite el guardar varios usuarios y no solo uno.

¿Cuáles son las desventajas de usar la local storage? Los datos son visibles desde el navegador en formato plano, las contraseñas no están encriptadas, cualquiera que

accediera al almacenamiento local podría modificarlos desde la consola, Estos datos se pueden eliminar accediendo directamente a borrar la memoria local o borrando la cache del navegador. Al usar localstorage en lugar de sessionStorage los datos no se eliminan automáticamente al cerrar el navegador.

El acceso a estos datos lo podemos hacer desde el propio navegador
F12-aplicación-almacenamiento



7.3 Uso de sessionStorage para el login component.

Muestra los datos del usuario logado; actualmente sólo la foto que le asigna por defecto y su nombre.

Recupera los datos desde sessionStorage lo que hace que una vez cerrado el navegador la sesión se cierre automáticamente.

Carga el componente profile con el nombre y la foto (avatar) asignado desde la sessionStorage.

7.4 Funcionamiento de la api implementada.

La Dog API es un servicio gratuito que proporciona imágenes aleatorias de perros y datos sobre diferentes razas. Ofrece la opción de listar todas las razas disponibles junto con información adicional, como su origen, características físicas y temperamento así como una imagen de la misma.

Para utilizar esta API, es necesario contar con una API Key, ya que se trata de una API protegida que requiere autenticación es decir no es una API pública si no que requiere que el usuario se registre y utilice un token de autenticación en cada solicitud. Esto permite controlar el acceso a los datos, evitar el uso no autorizado y establecer límites de solicitudes por usuario.

El servicio que implementa esta api es **dog.api.service.ts** que se encarga de hacer peticiones HTTP a The Dog API para obtener la lista de razas de perros. Podemos decir que este servicio es la "llave" para acceder a la API, ya que incluye la API Key necesaria para la autenticación. Llama a la Api consigue los datos y los devuelve en un observable a Angular.

Un observable no es un array aunque podamos confundirlo pues maneja datos en distintos momentos de tiempo, recibe datos de forma asíncrona, no es una lista de elementos en memoria si no un flujo. Cualquier componente es capaz de suscribirse a este observable (cualquiera porque el servicio esta inyectado en el root si no habría que importarlo para el que lo quiera usar).

En nuestro caso el componente que maneja este observable es **dog.breeds.component.ts**. Este componente se suscribe al Observable que devuelve la API y gestiona los datos recibidos. También realiza el filtrado que hemos establecido para mostrar los perros según su tamaño; calcula un peso medio y lo encuadra en la categoría que hemos asignado.

Una vez recibido los lanza a **dog.list.component.ts** que es el contenedor que recibe la lista de razas de perros, maneja la paginación y la muestra en la interfaz tal y como hemos establecido de ocho en ocho con el botón mostrar más si el usuario así lo desea. Además nos permite a través de un menú desplegable elegir el tamaño de las razas que queremos ver y nos muestra sólo las seleccionadas.

8. Conclusión

Mamás Perrunas representa una solución innovadora y en constante evolución para los amantes de los perros. A través del uso de tecnologías modernas como Angular, JSON, localStorage y API externas, se ha logrado un sitio web funcional, escalable y optimizado para la experiencia del usuario.

El enfoque modular basado en componentes Standalone ha permitido un desarrollo más limpio y eficiente, asegurando un rendimiento óptimo. Además, se han implementado medidas para mejorar la usabilidad, como la integración de menús intuitivos, un sistema de navegación lógico y el uso de imágenes generadas por inteligencia artificial para evitar problemas de derechos de autor.

A futuro, el proyecto puede expandirse mediante la implementación de un backend real, la optimización de la seguridad en la autenticación de usuarios y la ampliación de la comunidad mediante nuevas funcionalidades interactivas.